

Decisiones de diseño — Actividad 2

Identificación

- **Estudiante:** *(completar nombre)*
- **Actividad:** Pruebas unitarias, hooks y componentes en Task Manager
- **Proyecto:** Task Manager (Expo + React Native)
- **Repositorio (fork personal):** <https://github.com/Manzano-145/task-manager-testing-lab.git>
- **Rama:** actividad-2-pruebas
- **Fecha:** 31 de julio de 2026

Alcance de las pruebas

Funciones puras (12 pruebas)

Función	Archivo de prueba	Casos
validateTaskTitle	__tests__/utils/validateTask.test.ts	Títulos válidos, vacíos, espacios, longitud mínima/máxima
filterTasksByStatus	__tests__/utils/filterTasks.test.ts	pending, completed, all, archived, estado inválido
createTaskFromTitle	__tests__/utils/createTaskFromTitle.test.ts	Creación, trim, título vacío, id inválido

Hook (4 pruebas)

Hook	Archivo	Comportamientos
useCreateTask	__tests__/hooks/useCreateTask.test.ts	Estado inicial, submit exitoso, error, removeTask

Componentes (8+ pruebas)

Componente	Archivo	Comportamientos
TaskForm	__tests__/components/TaskForm.test.tsx	Render, onChangeText, submit válido, vacío, espacios
TaskCard	__tests__/components/TaskCard.test.tsx	Título, estados, onDelete

Selección de casos

- **Casos normales:** títulos válidos, filtrado por estado, creación de tareas.
- **Casos límite:** cadenas vacías, solo espacios, listas vacías, títulos > 100 caracteres.
- **Valores inesperados:** filtros inválidos (`toThrow`), identificadores vacíos.
- **Estados condicionales:** tarea pendiente vs completada en TaskCard.
- **Interacciones:** `fireEvent.press` , `fireEvent.changeText` , callbacks con `jest.fn()` .

Mocking

Dependencia	Motivo
taskService.createTask	Aislar el hook de la red; verificar transiciones de estado
jest.fn() en callbacks	Observar interacciones sin ejecutar lógica del padre
MSW (<code>src/mocks/</code>)	Simular API en pruebas de contrato e integración

Resultados

- **Suites:** 15 aprobadas / 15 totales.
- **Pruebas:** 58 aprobadas / 58 totales.
- **Cobertura:** 91.75% statements · 88.88% branches · 89.47% functions · 92.5% lines (meta interna: 70%).

La suite incluye pruebas unitarias, de hooks, componentes, integración, contrato y accesibilidad. Todas las pruebas pasan de forma consistente y sin dependencias externas reales (red, almacenamiento o reloj del sistema), gracias al aislamiento con mocks y MSW descrito arriba.

Conclusión: el proyecto supera el mínimo de 14 pruebas exigido por la actividad (58 implementadas) y las metas internas de cobertura, validando tanto la lógica de negocio pura como el comportamiento observable de hooks y componentes de interfaz.